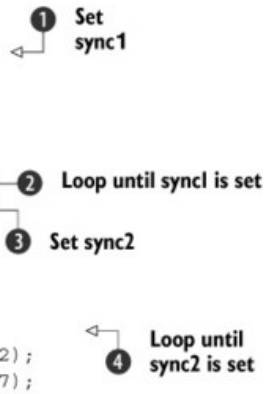
```
std::atomic<int> data[5];
std::atomic<bool> sync1(false),sync2(false);

void thread_1()
{
    data[0].store(42,std::memory_order_relaxed);
    data[1].store(97,std::memory_order_relaxed);
    data[2].store(17,std::memory_order_relaxed);
    data[3].store(-141,std::memory_order_relaxed);
    data[4].store(2003,std::memory_order_relaxed);
    sync1.store(true,std::memory_order_release);
}

void thread_2()
{
    while(!sync1.load(std::memory_order_acquire));
    sync2.store(true,std::memory_order_release);
}

void thread_3()
{
    while(!sync2.load(std::memory_order_acquire));
    assert(data[0].load(std::memory_order_relaxed)==42);
    assert(data[1].load(std::memory_order_relaxed)==97);
    assert(data[2].load(std::memory_order_relaxed)==17);
    assert(data[3].load(std::memory_order_relaxed)==-141);
    assert(data[4].load(std::memory_order_relaxed)==2003);
}
```



Even though `thread_2` only touches the variables `sync1` ❷ and `sync2` ❸, this is enough for synchronization between `thread_1` and `thread_3` to ensure that the `asserts` don't fire. First off, the stores to `data` from `thread_1` happens-before the store to `sync1` ❶, because they're sequenced-before it in the same thread. Because the load from `sync1` ❶ is in a `while` loop, it will eventually see the value stored from `thread_1` and thus form the second half of the release-acquire pair. Therefore, the store to `sync1` happens-before the final load from `sync1` in the `while` loop. This load is sequenced-before (and thus happens-before) the store to `sync2` ❸, which forms a release-acquire pair with the final load from the `while` loop in `thread_3` ❹. The store to `sync2` ❸ thus happens-before the load ❹, which happens-before the loads from `data`. Because of the transitive nature of happens-before, you can chain it all together: the stores to `data` happen-before the store to `sync1` ❶, which happens-before the load from `sync1` ❷, which happens-before the store to `sync2` ❸, which happens-before the load from `sync2` ❹, which happens-before the loads from `data`. Thus the stores to `data` in `thread_1` happen-before the loads from `data` in `thread_3`, and the `asserts` can't fire.

In this case, you could combine `sync1` and `sync2` into a single variable by using a read-modify-write operation with `memory_order_acq_rel` in `thread_2`. One option would be to use `compare_exchange_strong()` to ensure that the value is updated only once the store from `thread_1` has been seen:

```
std::atomic<int> sync(0);
void thread_1()
{
    // ...
    sync.store(1,std::memory_order_release);
}
void thread_2()
```